



Программирование на Python

Условные конструкции, строки, циклы

План лекции

1. Объекты в памяти
2. Изменяемые и неизменяемые типы объектов
3. Производительность при работе с памятью
4. Условные конструкции
5. Циклы
6. Сборщик мусора

Задача: собрать строку по частям

Насколько эффективен
этот код?

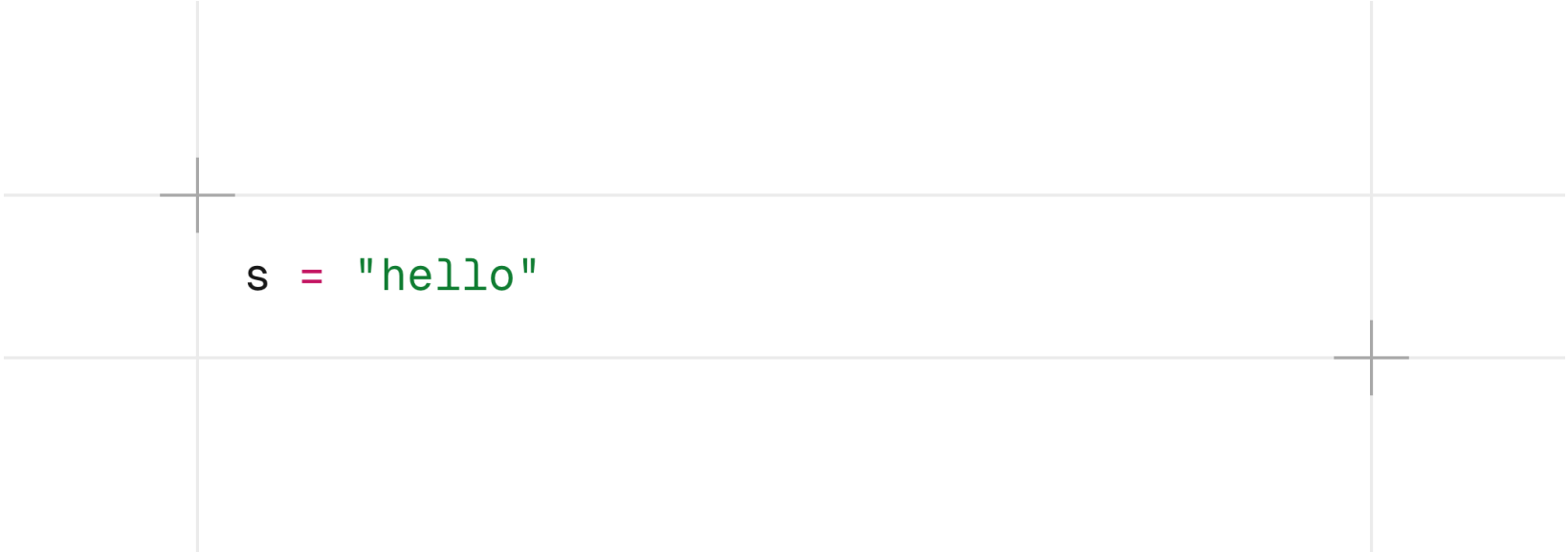
```
# Представим, что мы читаем файл по одному символу
# и хотим собрать все символы в одну большую строку.

# 1. Создаем пустую строку
result = ""

# 2. В цикле добавляем символы
result = result + "a"
result = result + "b"
result = result + "c"
# ... и так 100 000 раз ...
```

Что на самом деле
происходит в памяти
каждый раз, когда мы
выполняем **+**?

Самая важная операция в Python



```
s = "hello"
```

Появление объекта

Имена	Объекты
	Object type: str value: "hello"

Уникальный идентификатор

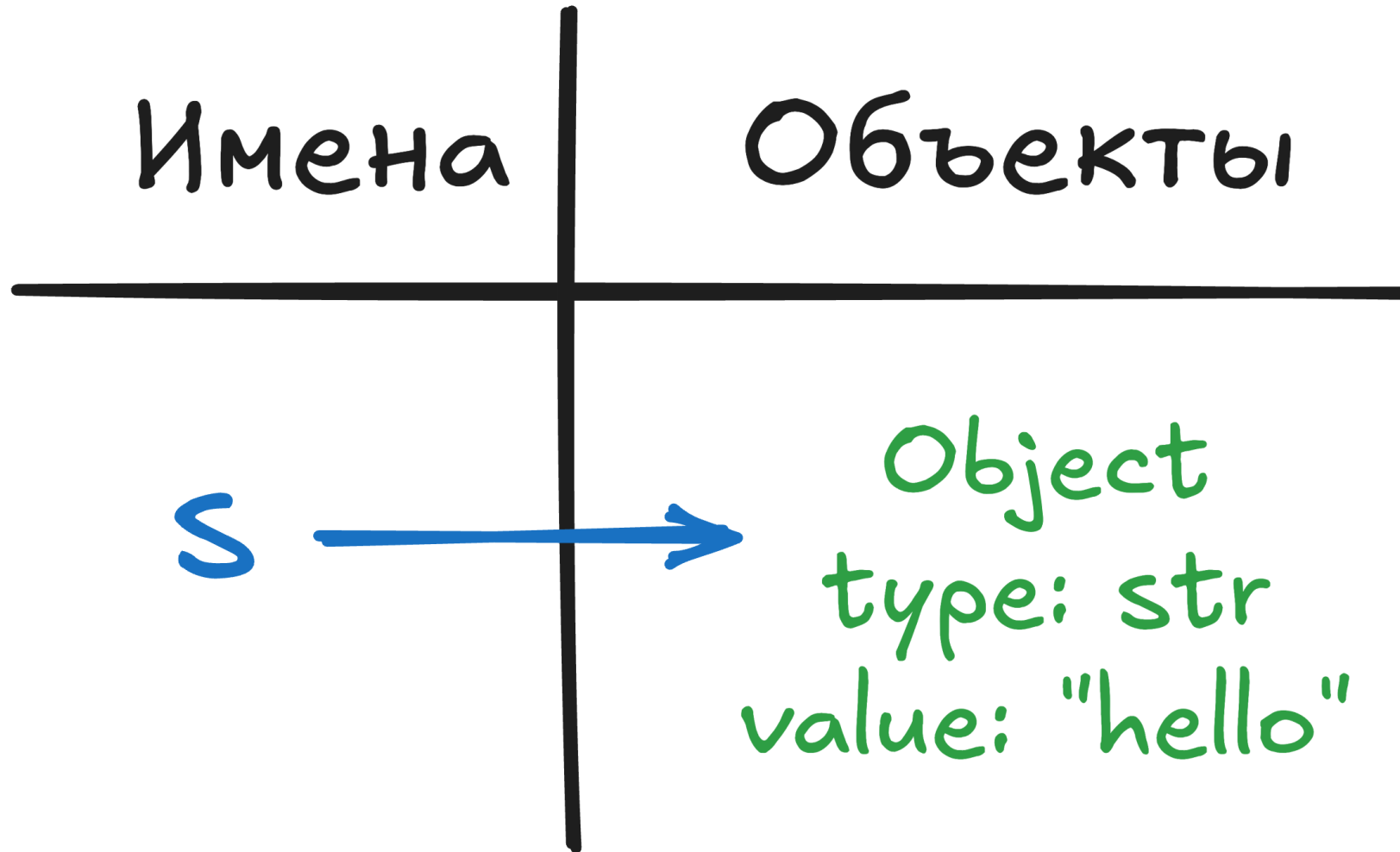
Каждый объект получает уникальный адрес в памяти. Мы можем увидеть его с помощью функции `id()`.

```
# Этот номер будет разным при каждом запуске
print(id("hello"))
# Вывод: 2194830113264
```

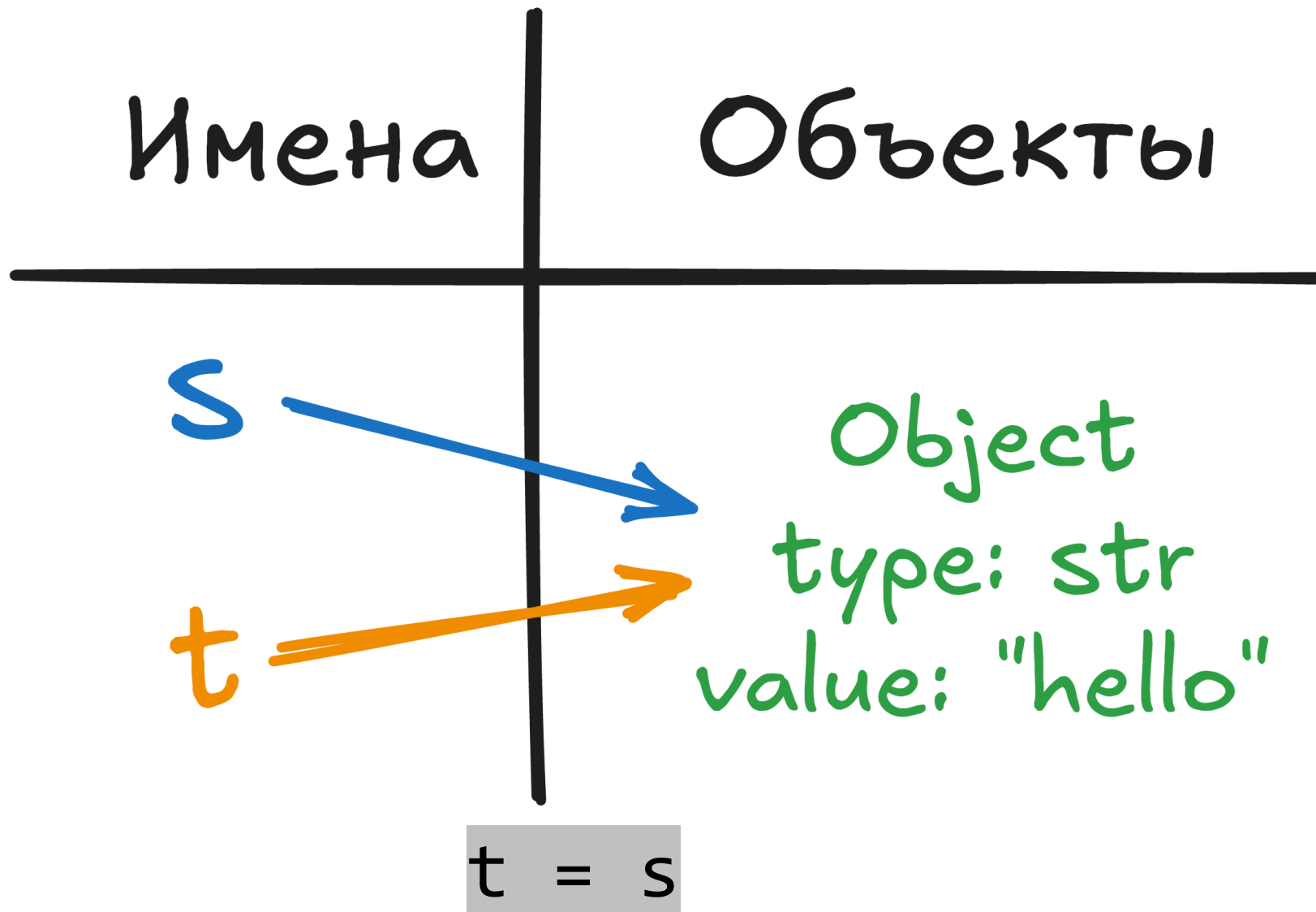
Появление имени

Имена	Объекты
<code>s</code>	<code>Object</code> <code>type: str</code> <code>value: "hello"</code>

Переменная = Ссылка



Не копирование, а ссылка



Не копирование, а ссылка

```
s = "hello"  
t = s  
print(f"ID s: {id(s)}")  
print(f"ID t: {id(t)}") # ID будут одинаковыми!
```

Output:

ID s: 4376955040

ID t: 4376955040

Как сравнивать? `is` vs `==`

- **`is`** (Идентичность): Проверяет, что два имени указывают на один и тот же объект. Сравнивает их `id()`.

➤ *Вопрос: "Это два ярлыка на одном и том же объекте?"*

- **`==`** (Равенство): Проверяет, что значения объектов равны.

➤ *Вопрос: »Значения объектов, на которые указывают эти ярлыки, одно и то же?«*

Проверяем наши ярлыки

```
s = "hello"
t = s # t - это просто второй ярлык для того же объекта

# Сравниваем значения
print(f"s == t: {s == t}")
# Вывод: True, потому что значения "hello" и "hello" равны

# Сравниваем идентичность объектов
print(f"s is t: {s is t}")
# Вывод: True, потому что s и t - имена одного и того же объекта

Output:
s == t: True
s is t: True
```

Одинаковые значения, разные объекты

```
# .strip() создает НОВЫЙ объект-строку
s1 = "  hello  ".strip()
s2 = "hello"

# Сравниваем значения
print(f"s1 == s2: {s1 == s2}")
# Вывод: True, потому что значения "hello" и "hello" равны

# Сравниваем идентичность объектов
print(f"s1 is s2: {s1 is s2}")
# Вывод: False, потому что это два разных объекта в памяти!

print(f"ID s1: {id(s1)}")
print(f"ID s2: {id(s2)}") # ID разные!
```

Output:

```
s1 == s2: True
s1 is s2: False
ID s1: 4400773232
ID s2: 4376955040
```

Неожиданное поведение

```
# Случай 1
```

```
a = 256
```

```
b = 256
```

```
print(f"a is b: {a is b}") # ???
```

```
# Случай 2
```

```
x = 257
```

```
y = 257
```

```
print(f"x is y: {x is y}") # ???
```

"Под капотом": кэширование целых чисел

- [-5; 256]
- Когда мы пишем `a = 256` и `b = 256`, Python просто дает нам ссылки на **уже существующий** объект. Поэтому `a is b` возвращает `True`.
- Число 257 выходит за этот диапазон, поэтому для `x` и `y` создаются **два разных объекта**. Поэтому `x is y` возвращает `False`.

Золотое правило

- Всегда используйте `==` для сравнения значений.
- Используйте `is` только для проверки, что объект является `None` (`if my_var is None:`).

Виды объектов

- **изменяемые** (`mutable`) и **неизменяемые** (`immutable`)
- Значение, хранящееся в объекте неизменяемого типа, невозможно изменить никакими способами. Единственный способ получить объект такого типа с другим значением — создать новый объект с нужным значением (`str`, `bytes`, `int`, `float`, `bool`, `tuple`).
- Значение, хранящееся в объекте изменяемого типа, можно изменить не создавая новый объект. Изменения объекта возможно за счет вызова его методов или применения к нему операторов (`list`, `dict`, `set`).

Строки (str)

```
my_string = "python"  
# Попробуем сделать первую букву заглавной  
my_string[0] = "P"
```

Output:

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[7], line 3  
      1 my_string = "python"  
      2 # Попробуем сделать первую букву заглавной  
----> 3 my_string[0] = "P"  
  
TypeError: 'str' object does not support item assignment
```

"Изменение" — это создание нового объекта

- Как же работает `+`?
- Когда мы пишем `s = s + "!"`, мы не меняем старую строку. Мы создаем новую и переклеиваем на нее ярлык `s`.

```
s = "hello"
print(f"ID до: {id(s)}")

s = s + "!"
print(f"ID после: {id(s)}") # ID изменился!
```

Output:
ID до: 4376955040
ID после: 4403284224

Что можно делать со строками

- Мы используем методы, которые не меняют исходную строку, а возвращают новую, измененную копию.
- Ключевая идея: исходная_строка -> метод() -> новая_строка

Смена регистра и удаление пробелов

```
s = "  Hello World!  "

# .upper() -> новая строка в верхнем регистре
upper_string = s.upper() # "  HELLO WORLD!  "

# .lower() -> новая строка в нижнем регистре
lower_string = s.lower() # "  hello world!  "

# .strip() -> новая строка без лишних пробелов по краям
clean_string = s.strip() # "Hello World!"
```

Исходник в безопасности

```
s = "    Hello World!    "
upper_string = s.upper()
lower_string = s.lower()
clean_string = s.strip()

print(f"Оригинал: '{s}'")
print(f"ID оригинала: {id(s)}")
print(f"ID upper_string: {id(upper_string)}") # Другой ID
print(f"ID lower_string: {id(lower_string)}") # Другой ID
print(f"ID clean_string: {id(clean_string)}") # Другой ID
```

Output:

```
Оригинал: '    Hello World!    '
ID оригинала: 4406343408
ID upper_string: 4406338160
ID lower_string: 4414822000
ID clean_string: 4414926640
```

Замена частей строки

```
sentence = "Я изучаю язык Java"

# .replace('старое', 'новое') -> возвращает новую строку
correct_sentence = sentence.replace("Java", "Python")

print(correct_sentence) # "Я изучаю язык Python"
print(sentence)        # "Я изучаю язык Java"
```

Output:

Я изучаю язык Python

Я изучаю язык Java

Проверка начала и конца строки

```
file_name = "document.pdf"

# .startswith('подстрока')
print(file_name.startswith("doc")) # True

# .endswith('подстрока')
print(file_name.endswith(".pdf")) # True
print(file_name.endswith(".txt")) # False
```

Output:

```
True
True
False
```

Байт-строки (bytes)

- `str` – это текст для людей (символы Unicode).
- `bytes` – это "сырые" данные для машин (числа от 0 до 255).
- Тоже **неизменяемы**.
- Используются для работы с файлами, сетью, шифрованием.

Байт-строки (bytes)

```
my_bytes = b'hello'  
print(type(my_bytes))  
print(my_bytes)  
print(list(my_bytes))
```

Output:

```
<class 'bytes'>  
b'hello'  
[104, 101, 108, 108, 111]
```

Преобразование текста в байты

```
# str -> bytes (записываем в файл)
text = "привет"
encoded = text.encode('utf-8')
print(encoded) # b'\xd0\xbf\xd1\x80...'
```

```
# bytes -> str (читаем из файла)
decoded = encoded.decode('utf-8')
print(decoded) # 'привет'
```

Output:

```
b'\xd0\xbf\xd1\x80\xd0\xb8\xd0\xb2\xd0\xb5\xd1\x82'
привет
```

Задача: собрать строку по частям

Насколько эффективен
этот код?

```
# Представим, что мы читаем файл по одному символу
# и хотим собрать все символы в одну большую строку.

# 1. Создаем пустую строку
result = ""

# 2. В цикле добавляем символы
result = result + "a"
result = result + "b"
result = result + "c"
# ... и так 100 000 раз ...
```

Что на самом деле
происходит в памяти
каждый раз, когда мы
выполняем **+**?

Принятие решений

- Конструкции `if` и `else` позволяют выполнять разный код в зависимости от условия.
- В Python блоки кода выделяются **отступами**.

```
temperature = 15
if temperature > 20:
    print("Наденьте футболку")
else:
    print("Лучше взять куртку")
```

Цепочка условий

- `elif`

```
day = "вторник"
if day == "понедельник":
    print("Тяжелый день")
elif day == "вторник":
    print("Уже лучше")
else:
    print("Скоро выходные!")
```

"Истинность" строк

- `if` может проверять не только сравнения, но и сами строки.
 - Пустая строка `""` – это **Ложь**.
 - Любая **непустая** строка – это **Истина**.

```
# "Pythonic" way - элегантный способ
user_name = input("Введите ваше имя: ")

if user_name:
    print(f"Привет, {user_name}!")
else:
    print("Привет, незнакомец!")
```

Составные условия

- Составные условия можно записывать с выражениями, включающими логические операции `and` и `or`.
 - В выражении `x and y` сначала вычисляется значение `x` и только если оно истинно (`True`), то вычисляется `y`;
 - В выражении `x or y` сначала вычисляется значение `x` и только если оно ложно (`False`), то вычисляется `y`;

```
if x != 0 and y / x > 0:  
    print("do something")
```

```
if x == 0 or y / x > 0:  
    print("do something")
```

Современный способ ветвления `match`

- Появился в Python 3.10
- Похож на оператор `switch`

```
match значение:  
    case шаблон_1:  
        инструкция  
    case шаблон_2:  
        инструкция  
    case _:  
        инструкция по умолчанию
```

Современный способ ветвления

match

```
http_status = 404

if http_status == 200:
    print("OK")
elif http_status == 404:
    print("Not Found")
elif http_status == 500:
    print("Server Error")
else:
    print("Unknown Status")
```

```
http_status = 404

match http_status:
    case 200:
        print("OK")
    case 404:
        print("Not Found")
    case 500:
        print("Server Error")
    case _:
        print("Unknown Status")
```

Современный способ ветвления

match

```
# Гораздо чище, чем if status == 401 or status == 403 or ...
match http_status:
    case 200 | 201:
        print("Успех")
    case 401 | 403 | 404:
        print("Ошибка на стороне клиента")
    case 500 | 503:
        print("Ошибка на стороне сервера")
```

Output:

Ошибка на стороне клиента

`match` или `if/elif/else`

- **Используйте `match`, когда:**

1. У вас есть одна переменная, которую вы сравниваете с несколькими конкретными значениями.
2. Логика сводится к вопросу: "Какова форма или значение этих данных?".
3. Цепочка `if/elif/else` становится длинной и однообразной.

- **Оставайтесь с `if/elif/else`, когда:**

1. Условия сложные и затрагивают несколько разных переменных (`if user.is_active and not user.is_banned:`).
2. Вы проверяете диапазоны или неравенства (`if age > 18:`).
3. У вас всего 1-2 условия, для которых `match` будет избыточен.

Цикл `while`

```
while условие:  
    первая инструкция тела цикла  
    ...  
    последняя инструкция тела цикла
```

```
i = 1  
while i < 1000:  
    print(i)  
    i *= 2
```

Output:

```
1  
2  
4  
8  
16  
32  
64  
128  
256  
512
```

Ключевые слова

- `break` немедленно покидает цикл;
- `continue` прекращает исполнение тела цикла на данной итерации и сразу заходит на следующую.

```
i = 0
while True:
    i += 1

    if i == 4:
        print("skip", end=" ")
        continue
    elif i == 8:
        print("end")
        break

    print(i, end=" ")
```

Output:

```
1, 2, 3, skip, 5, 6, 7, end
```

Цикл `for`

```
for переменная in итерируемый_объект:  
    первая инструкция тела цикла  
    ...  
    последняя инструкция тела цикла
```

Цикл `for`

```
v = [0, 1, 2, 3, 4, 5]
for i in v:
    print(i, end=" ")
```

Output:

```
0 1 2 3 4 5
```

Диапазон range

- Один аргумент `range(n)` создаёт последовательность целых чисел от 0 включая до n не включая.

```
for i in range(5):  
    print(i, end=" ")
```

Output:

```
0 1 2 3 4
```

Диапазон range

- Два аргумента `range(start, stop)` - диапазон целых чисел от `start` включая до `stop` не включая.

```
for i in range(-4, 5):  
    print(i, end=" ")
```

Output:

```
-4 -3 -2 -1 0 1 2 3 4
```

Диапазон range

- Три аргумента `range(start, stop, step)` - то же, что и `range(start, stop)`, но с шагом в `step`.

```
for i in range(0, 10, 2):  
    print(i, end=" ")
```

Output:

```
0 2 4 6 8
```

Получение индекса

```
for index, char in enumerate("abc"):
    print(f"Индекс: {index}, Символ: {char}")
```

Output:

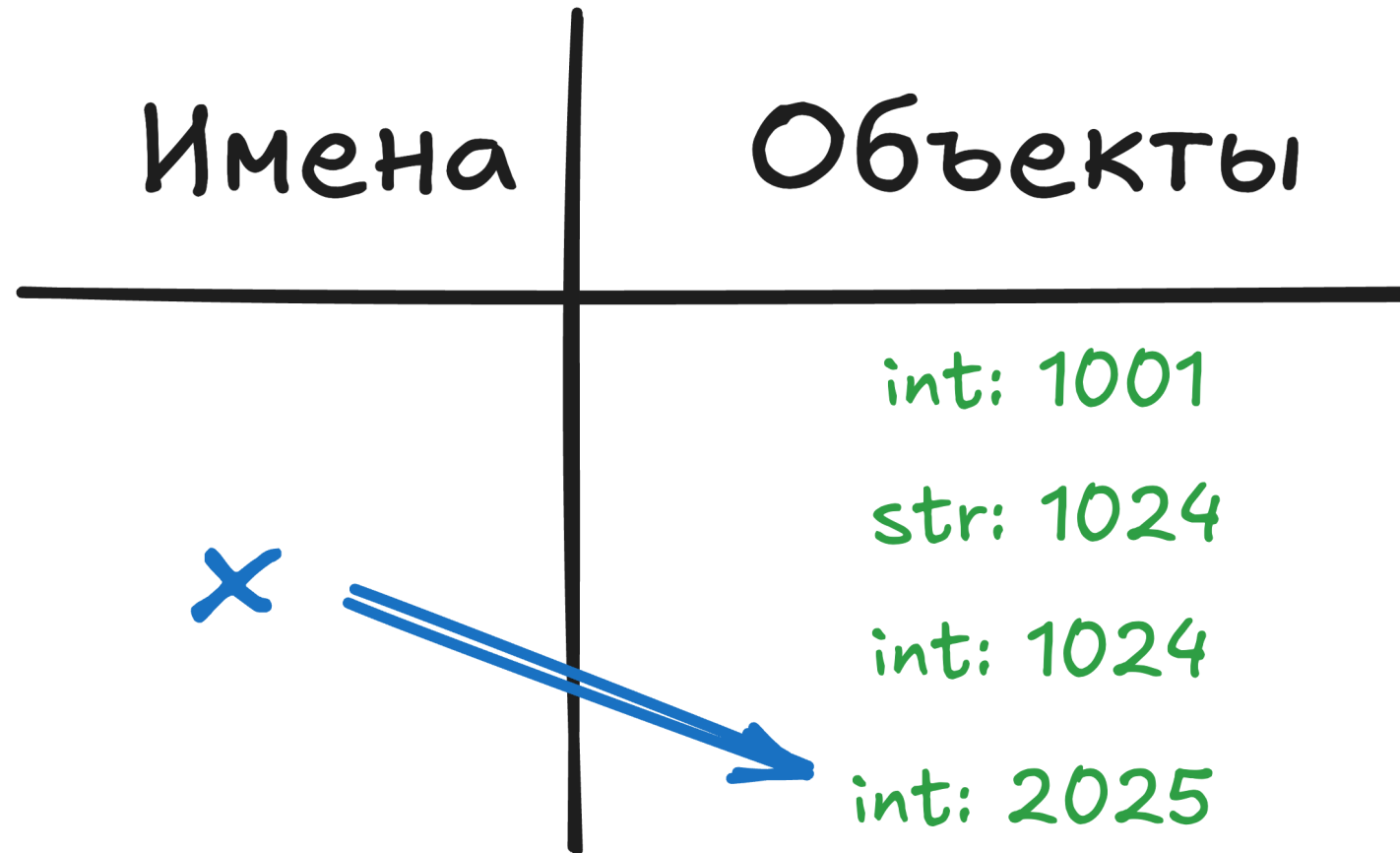
Индекс: 0, Символ: a

Индекс: 1, Символ: b

Индекс: 2, Символ: c

Сборка мусора

```
x = 1001 + int("1024")
```



Подсчет ссылок

- <https://github.com/python/cpython/blob/main/Include/object.h#L156>

```
struct _object {  
    // ob_tid stores the thread id (or zero). It is also used by the GC and the  
    // trashcan mechanism as a linked list pointer and by the GC to store the  
    // computed "gc_refs" refcount.  
    _Py_ALIGNED_DEF(_PyObject_MIN_ALIGNMENT, uintptr_t) ob_tid;  
    uint16_t ob_flags;  
    PyMutex ob_mutex;           // per-object lock  
    uint8_t ob_gc_bits;        // gc-related state  
    uint32_t ob_ref_local;      // local reference count  
    Py_ssize_t ob_ref_shared;   // shared (atomic) reference count  
    PyTypeObject *ob_type;  
};
```

Подсчет ссылок

```
import sys

my_obj = "a string"
# Обратите внимание: getrefcount() сам создает временную ссылку,
# поэтому результат всегда на 1 больше, чем мы ожидаем.
print(sys.getrefcount(my_obj)) # Вывод: 2

another_ref = my_obj
print(sys.getrefcount(my_obj)) # Вывод: 3

del another_ref
print(sys.getrefcount(my_obj)) # Вывод: 2
```

Output:

```
2
3
2
```

Циклические ссылки

```
# a ссылается на b, а b ссылается на a  
a = [...]  
b = [a]  
a.append(b)
```

```
# Удаляем внешние ссылки  
del a  
del b
```

Циклический Сборщик Мусора (Generational GC)

1. Это отдельный процесс, который запускается **периодически**, а не мгновенно.
2. Он не смотрит на счетчики. Вместо этого он ищет "острова" объектов, до которых **невозможно добраться** из основной части вашей программы.
3. Найдя такой "остров" (как в нашем примере с циклическими ссылками), он принудительно удаляет все объекты на нем.

Двухуровневая система

- **Подсчет ссылок:**
 - Роль: Основной механизм.
 - Плюсы: Быстрый, предсказуемый.
 - Минусы: Не справляется с циклическими ссылками.
- **Циклический Сборщик Мусора:**
 - Роль: Специализированный инструмент.
 - Плюсы: Находит и устраняет утечки памяти от циклических ссылок.
 - Минусы: Работает медленнее, запускается периодически.

Превью следующей лекции

- Списки
- Словари
- Кортежи
- Множества

Полезные материалы

- <https://docs.python.org/3/tutorial/controlflow.html>
- <https://nedbatchelder.com/text/names.html>
- <https://realpython.com/python-memory-management/>